

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

OPERATING SYSTEMS (23CS4PCOPS)

Submitted by

Priyanka Rajput(1BF24CS234)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under

VTU) BENGALURU-560019

February to June 2026

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum) Department
of Computer Science and Engineering**



This is to certify that the Lab work entitled **“OPERATING SYSTEMS – 23CS4PCOPS”** carried out by **Priyanka Rajput(1BF24CS234)**, who is Bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of OPERATING SYSTEMS- (23CS4PCOPS) work prescribed for the said degree.

Sandhya A Kulkarni

Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda

Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1.	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. (Any one) a) FCFS b) SJF c) Priority d) Round Robin (Experiment with different quantum sizes for RR algorithm)	5
2.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	25
3.	Write a C program to simulate Real-Time CPU Scheduling algorithms: (Any one) a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling	29
4	Write a C program to simulate: (Any one) a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem	35
5	Write a C program to simulate: (Any one) a) Bankers' algorithm for the purpose of deadlock avoidance. b) Deadlock Detection	42
6	Write a C program to simulate the following contiguous memory allocation techniques. (Any one) a) Worst-fit b) Best-fit c) First-fit	50
7	Write a C program to simulate page replacement algorithms. (Any one) a) FIFO b) LRU c) Optimal	56

Course outcomes:

CO1	Apply the different concepts and functionalities of Operating System.
CO2	Analyse various Operating system strategies and techniques
CO3	Demonstrate the different functionalities of Operating System.
CO4	Conduct practical experiments to implement the functionalities of Operating system.

Lab program 1:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. (Any one)

- a) FCFS
- b) SJF
- c) Priority
- d) Round Robin (Experiment with different quantum sizes for RR algorithm)

Code in C FCFS

```
#include <stdio.h>

int main(){

    int n,i;
    int at[20],bt[20],ct[20],tat[20],wt[20],rt[20];
    float avg_wt=0,avg_tat=0,avg_rt=0;

    printf("enter number of processes");
    scanf("%d",&n);

    printf("enter arrival time and burst time\n");

    for(int i=0;i<n;i++){
        printf("P%d AT:",i+1);
        scanf("%d",&at[i]);
        printf("P%d BT:",i+1);
        scanf("%d",&bt[i]);
    }

    ct[0]=bt[0]+at[0];

    for(int i=1;i<n;i++){
        if(ct[i-1]<at[i]){
            ct[i]=at[i]+bt[i];
        }
    }
}
```

```

    else{
        ct[i]=ct[i-1]+bt[i];
    }
}

for(i=0;i<n;i++){
    tat[i]=ct[i]-at[i];
    wt[i]=tat[i]-bt[i];
    rt[i]=wt[i];

    avg_wt+=wt[i];
    avg_tat+=tat[i];
    avg_rt+=rt[i];
}
printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\tRT\n");

for(int i=0;i<n;i++){
    printf("\nP%d\t%d\t%d\t%d\t%d\t%d\t%d", i+1,at[i],bt[i],ct[i],tat[i],wt[i],rt[i]);
}

printf("\nAverage waiting time= %.2f",avg_wt/n);
printf("\nAverage turnaround time= %.2f",avg_tat/n);
printf("\nAverage response time= %.2f",avg_rt/n);

return 0;
}

```

Output

```
"C:\Users\Priyanka rajput\One" X + v
enter number of processes3
enter arrival time and burst time
P1 AT:0
P1 BT:5
P2 AT:1
P2 BT:3
P3 AT:2
P3 BT:4

Process AT      BT      CT      TAT      WT      RT
P1      0      5      5      5      0      0
P2      1      3      8      7      4      4
P3      2      4     12     10      6      6
Average waiting time= 3.33
Average turnaround time= 7.33
Average response time= 3.33
Process returned 0 (0x0)  execution time : 12.908 s
Press any key to continue.
```

Code in C SJF(non preemptive)

```
#include <stdio.h>
#include <limits.h>

int main() {
    int n, i, time = 0, completed = 0;
    int at[20], bt[20], ct[20], tat[20], wt[20], rt[20];
    int done[20] = {0};

    float avg_wt = 0, avg_tat = 0, avg_rt = 0;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    printf("Enter Arrival Time and Burst Time:\n");

    for(i=0;i<n;i++){
        printf("P%d AT: ",i+1);
        scanf("%d",&at[i]);

        printf("P%d BT: ",i+1);
        scanf("%d",&bt[i]);
    }

    while(completed < n){

        int shortest = -1;
        int min_bt = INT_MAX;

        for(i=0;i<n;i++){
            if(at[i] <= time && done[i]==0 && bt[i] < min_bt){
                min_bt = bt[i];
                shortest = i;
            }
        }

        time += min_bt;
        done[shortest] = 1;
        completed++;
        avg_wt += wt[shortest];
        avg_tat += tat[shortest];
        avg_rt += rt[shortest];
    }

    printf("Average Waiting Time: %.2f\n", avg_wt/n);
    printf("Average Turnaround Time: %.2f\n", avg_tat/n);
    printf("Average Response Time: %.2f\n", avg_rt/n);
}
```



```

    if(shortest == -1){
        time++;
        continue;
    }

    rt[shortest] = time - at[shortest];

    time += bt[shortest];
    ct[shortest] = time;

    tat[shortest] = ct[shortest] - at[shortest];
    wt[shortest] = tat[shortest] - bt[shortest];

    done[shortest] = 1;
    completed++;

    avg_wt += wt[shortest];
    avg_tat += tat[shortest];
    avg_rt += rt[shortest];
}

printf("\nProcess AT BT CT TAT WT RT\n");

for(i=0;i<n;i++){
    printf("P%d    %d %d %d %d %d %d\n",
        i+1,at[i],bt[i],ct[i],tat[i],wt[i],rt[i]);
}

printf("\nAverage Waiting Time = %.2f",avg_wt/n);
printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
printf("\nAverage Response Time = %.2f\n",avg_rt/n);

return 0;
}

```

OUTPUT

```
"C:\Users\Priyanka rajput\One" × + v
Enter number of processes: 3
Enter Arrival Time and Burst Time:
P1 AT: 0
P1 BT: 5
P2 AT: 1
P2 BT: 3
P3 AT: 2
P3 BT: 4

Process AT BT CT TAT WT RT
P1      0  5  5  5  0  0
P2      1  3  8  7  4  4
P3      2  4 12 10  6  6

Average Waiting Time = 3.33
Average Turnaround Time = 7.33
Average Response Time = 3.33

Process returned 0 (0x0)  execution time : 14.499 s
Press any key to continue.
```

Code in C SJF(preemptive)

```
#include <stdio.h>

#include <limits.h>

int main(){

    int n,i,time=0,completed=0;

    int at[20], bt[20], rem_bt[20];

    int ct[20], tat[20], wt[20], rt[20];

    int first[20]={0};

    float avg_wt=0, avg_tat=0, avg_rt=0;

    printf("Enter number of processes: ");

    scanf("%d",&n);

    printf("Enter Arrival Time and Burst Time:\n");

    for(i=0;i<n;i++){

        printf("P%d AT: ",i+1);

        scanf("%d",&at[i]);

        printf("P%d BT: ",i+1);

        scanf("%d",&bt[i]);

        rem_bt[i]=bt[i];

        rt[i]=-1;

    }
```

```

while(completed<n){

    int shortest=-1;

    int min_bt=INT_MAX;

    for(i=0;i<n;i++){

        if(at[i]<=time && rem_bt[i]>0 && rem_bt[i]<min_bt){

            min_bt=rem_bt[i];

            shortest=i;

        }

    }

    if(shortest==-1){

        time++;

        continue;

    }

    if(rt[shortest]==-1)

        rt[shortest]=time-at[shortest];

    rem_bt[shortest]--;

    time++;

    if(rem_bt[shortest]==0){

        completed++;

        ct[shortest]=time;

        tat[shortest]=ct[shortest]-at[shortest];

```

```

        wt[shortest]=tat[shortest]-bt[shortest];

        avg_wt+=wt[shortest];
        avg_tat+=tat[shortest];
        avg_rt+=rt[shortest];
    }
}

printf("\nProcess AT BT CT TAT WT RT\n");

for(i=0;i<n;i++){
    printf("P%d    %d %d %d %d %d %d\n",
        i+1,at[i],bt[i],ct[i],tat[i],wt[i],rt[i]);
}

printf("\nAverage Waiting Time = %.2f",avg_wt/n);
printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
printf("\nAverage Response Time = %.2f\n",avg_rt/n);

return 0;
}

```

OUTPUT

```
"C:\Users\Priyanka rajput\One" × + v
Enter number of processes: 3
Enter Arrival Time and Burst Time:
P1 AT: 0
P1 BT: 5
P2 AT: 1
P2 BT: 3
P3 AT: 2
P3 BT: 4

Process AT BT CT TAT WT RT
P1      0  5  8   8   3   0
P2      1  3  4   3   0   0
P3      2  4 12  10   6   6

Average Waiting Time = 3.00
Average Turnaround Time = 7.00
Average Response Time = 2.00

Process returned 0 (0x0)    execution time : 15.280 s
Press any key to continue.
```

Code in C Priority(Preemptive)

```
#include <stdio.h>
#include <limits.h>

int main()
{
    int n,i,time=0,completed=0;
    int at[20],bt[20],pr[20],rem_bt[20];
    int ct[20],tat[20],wt[20],rt[20];

    float avg_wt=0,avg_tat=0,avg_rt=0;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    printf("Enter Arrival Time, Burst Time and Priority:\n");

    for(i=0;i<n;i++)
    {
        printf("P%d AT: ",i+1);
        scanf("%d",&at[i]);

        printf("P%d BT: ",i+1);
        scanf("%d",&bt[i]);

        printf("P%d Priority: ",i+1);
        scanf("%d",&pr[i]);

        rem_bt[i]=bt[i];
        rt[i]=-1;
    }

    while(completed<n)
    {
        int highest=-1;
        int min_pr=INT_MAX;

        for(i=0;i<n;i++)
        {
            if(at[i]<=time && rem_bt[i]>0 && pr[i]<min_pr)
            {
                min_pr=pr[i];
                highest=i;
            }
        }
    }
```

```

if(highest==-1)
{
    time++;
    continue;
}

if(rt[highest]==-1)
    rt[highest] = time - at[highest];

rem_bt[highest]--;
time++;

if(rem_bt[highest]==0)
{
    completed++;

    ct[highest]=time;
    tat[highest]=ct[highest]-at[highest];
    wt[highest]=tat[highest]-bt[highest];

    avg_wt+=wt[highest];
    avg_tat+=tat[highest];
    avg_rt+=rt[highest];
}
}

printf("\nProcess AT BT PR CT TAT WT RT\n");

for(i=0;i<n;i++)
{
    printf("P%d    %d %d %d %d %d %d %d\n",
        i+1,at[i],bt[i],pr[i],ct[i],tat[i],wt[i],rt[i]);
}

printf("\nAverage Waiting Time = %.2f",avg_wt/n);
printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
printf("\nAverage Response Time = %.2f\n",avg_rt/n);

return 0;
}

```


OUTPUT

```
"C:\Users\Priyanka rajput\One" X + v
Enter number of processes: 4
Enter Arrival Time, Burst Time and Priority:
P1 AT: 0
P1 BT: 5
P1 Priority: 2
P2 AT: 1
P2 BT: 3
P2 Priority: 1
P3 AT: 2
P3 BT: 4
P3 Priority: 3
P4 AT: 3
P4 BT: 2
P4 Priority: 2

Process AT BT PR CT TAT WT RT
P1      0  5  2  8  8   3  0
P2      1  3  1  4  3   0  0
P3      2  4  3  14 12   8  8
P4      3  2  2  10  7   5  5

Average Waiting Time = 4.00
Average Turnaround Time = 7.50
Average Response Time = 3.25

Process returned 0 (0x0)   execution time : 37.332 s
Press any key to continue.
```

Code in C Priority(Non-Preemptive)

```
#include <stdio.h>
#include <limits.h>

int main()
{
    int n,i,time=0,completed=0;
    int at[20],bt[20],pr[20];
    int ct[20],tat[20],wt[20],rt[20];
    int done[20]={0};

    float avg_wt=0,avg_tat=0,avg_rt=0;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    printf("Enter Arrival Time, Burst Time and Priority:\n");

    for(i=0;i<n;i++)
    {
        printf("P%d AT: ",i+1);
        scanf("%d",&at[i]);

        printf("P%d BT: ",i+1);
        scanf("%d",&bt[i]);

        printf("P%d Priority: ",i+1);
        scanf("%d",&pr[i]);
    }

    while(completed<n)
    {
        int highest=-1;
        int min_pr=INT_MAX;

        for(i=0;i<n;i++)
        {
            if(at[i]<=time && done[i]==0 && pr[i]<min_pr)
```

```

        {
            min_pr=pr[i];
            highest=i;
        }
    }

    if(highest==-1)
    {
        time++;
        continue;
    }

    rt[highest] = time - at[highest];

    time += bt[highest];
    ct[highest] = time;

    tat[highest] = ct[highest] - at[highest];
    wt[highest] = tat[highest] - bt[highest];

    avg_wt += wt[highest];
    avg_tat += tat[highest];
    avg_rt += rt[highest];

    done[highest]=1;
    completed++;
}

printf("\nProcess AT BT PR CT TAT WT RT\n");

for(i=0;i<n;i++)
{
    printf("P%d    %d %d %d %d %d %d %d\n",
        i+1,at[i],bt[i],pr[i],ct[i],tat[i],wt[i],rt[i]);
}

printf("\nAverage Waiting Time = %.2f",avg_wt/n);
printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
printf("\nAverage Response Time = %.2f\n",avg_rt/n);

```

```
return 0;
}
```

OUTPUT

```
"C:\Users\Priyanka rajput\One" × + ~
Enter number of processes: 4
Enter Arrival Time, Burst Time and Priority:
P1 AT: 0
P1 BT: 5
P1 Priority: 2
P2 AT: 1
P2 BT: 3
P2 Priority: 1
P3 AT: 2
P3 BT: 4
P3 Priority: 3
P4 AT: 3
P4 BT: 2
P4 Priority: 2

Process AT BT PR CT TAT WT RT
P1      0  5  2  5  5  0  0
P2      1  3  1  8  7  4  4
P3      2  4  3  14 12  8  8
P4      3  2  2  10  7  5  5

Average Waiting Time = 4.25
Average Turnaround Time = 7.75
Average Response Time = 4.25

Process returned 0 (0x0)   execution time : 27.747 s
Press any key to continue.
```

Code in c round robin

```
#include <stdio.h>

int main() {
    int n, tq;
    int at[20], bt[20], rem_bt[20];
    int ct[20], tat[20], wt[20], rt[20];
    int completed = 0, time = 0;

    float avg_wt = 0, avg_tat = 0, avg_rt = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter Time Quantum: ");
    scanf("%d", &tq);

    printf("Enter Arrival Time and Burst Time:\n");

    for(int i = 0; i < n; i++) {
        printf("P%d AT: ", i + 1);
        scanf("%d", &at[i]);

        printf("P%d BT: ", i + 1);
        scanf("%d", &bt[i]);

        rem_bt[i] = bt[i];
        rt[i] = -1;
    }

    while(completed < n) {

        int found = 0;

        for(int i = 0; i < n; i++) {

            if(at[i] <= time && rem_bt[i] > 0) {
```

```

        found = 1;

        if(rt[i] == -1)
            rt[i] = time - at[i];

        if(rem_bt[i] > tq) {
            time += tq;
            rem_bt[i] -= tq;
        }
        else {
            time += rem_bt[i];

            ct[i] = time;
            tat[i] = ct[i] - at[i];
            wt[i] = tat[i] - bt[i];

            avg_wt += wt[i];
            avg_tat += tat[i];
            avg_rt += rt[i];

            rem_bt[i] = 0;
            completed++;
        }
    }

    if(!found)
        time++;
}

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\tRT\n");

for(int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i + 1, at[i], bt[i], ct[i],
        tat[i], wt[i], rt[i]);
}

```

```
printf("\nAverage Waiting Time = %.2f", avg_wt / n);  
    printf("\nAverage Turnaround Time = %.2f", avg_tat / n);  
    printf("\nAverage Response Time = %.2f\n", avg_rt / n);  
  
    return 0;
```

OUTPUT

```
"C:\Users\Priyanka rajput\One" × + v
Enter number of processes: 3
Enter Time Quantum: 2
Enter Arrival Time and Burst Time:
P1 AT: 0
P1 BT: 5
P2 AT: 1
P2 BT: 3
P3 AT: 2
P3 BT: 1

Process AT      BT      CT      TAT      WT      RT
P1      0      5      9      9      4      0
P2      1      3      8      7      4      1
P3      2      1      5      3      2      2

Average Waiting Time = 3.33
Average Turnaround Time = 6.33
Average Response Time = 1.00

Process returned 0 (0x0)   execution time : 23.120 s
Press any key to continue.
```

```
"C:\Users\Priyanka rajput\One" × + v
Enter number of processes: 3
Enter Time Quantum: 3
Enter Arrival Time and Burst Time:
P1 AT: 0
P1 BT: 5
P2 AT: 1
P2 BT: 3
P3 AT: 2
P3 BT: 1

Process AT      BT      CT      TAT      WT      RT
P1      0      5      9      9      4      0
P2      1      3      6      5      2      2
P3      2      1      7      5      4      4

Average Waiting Time = 3.33
Average Turnaround Time = 6.33
Average Response Time = 2.00

Process returned 0 (0x0)   execution time : 22.148 s
Press any key to continue.
```


Lab program 2:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories –system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

Code in C

```
#include <stdio.h>

#define MAX 20

int main() {
    int n, i, time = 0, completed = 0;

    int at[MAX], bt[MAX], rt[MAX], ct[MAX], wt[MAX], tat[MAX];
    int type[MAX]; // 0 = system, 1 = user

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        printf("\nProcess %d\n", i);

        printf("Enter arrival time: ");
        scanf("%d", &at[i]);

        printf("Enter burst time: ");
        scanf("%d", &bt[i]);

        printf("Enter type (0 = System, 1 = User): ");
        scanf("%d", &type[i]);

        rt[i] = bt[i];
    }
}
```

```

int done[MAX] = {0};

while(completed < n) {

    int current = -1;

    // Step 1: Check system processes first
    for(i = 0; i < n; i++) {
        if(at[i] <= time && rt[i] > 0 && type[i] == 0) {
            current = i;
            break; // FCFS among system
        }
    }

    // Step 2: If no system process, check user
    if(current == -1) {
        for(i = 0; i < n; i++) {
            if(at[i] <= time && rt[i] > 0 && type[i] == 1) {
                current = i;
                break; // FCFS among user
            }
        }
    }

    // Step 3: CPU idle
    if(current == -1) {
        time++;
        continue;
    }

    // Step 4: Execute 1 unit

```

```

    rt[current]--;
    time++;

    // Step 5: Completion
    if(rt[current] == 0) {
        ct[current] = time;
        tat[current] = ct[current] - at[current];
        wt[current] = tat[current] - bt[current];

        completed++;
    }
}

float avg_wt = 0, avg_tat = 0;

printf("\nID\tType\tAT\tBT\tCT\tWT\tTAT\n");

for(i = 0; i < n; i++) {
    printf("%d\t%s\t%d\t%d\t%d\t%d\t%d\n",
        i,
        (type[i] == 0) ? "System" : "User",
        at[i], bt[i], ct[i], wt[i], tat[i]);

    avg_wt += wt[i];
    avg_tat += tat[i];
}

printf("\nAverage Waiting Time: %.2f", avg_wt/n);
printf("\nAverage Turnaround Time: %.2f\n", avg_tat/n);

return 0;
}

```

OUTPUT

```
"C:\Users\Priyanka rajput\One" × + ∨  
Enter number of processes: 4  
  
Process 0  
Enter arrival time: 0  
Enter burst time: 7  
Enter type (0 = System, 1 = User): 1  
  
Process 1  
Enter arrival time: 1  
Enter burst time: 5  
Enter type (0 = System, 1 = User): 0  
  
Process 2  
Enter arrival time: 2  
Enter burst time: 3  
Enter type (0 = System, 1 = User): 1  
  
Process 3  
Enter arrival time: 3  
Enter burst time: 4  
Enter type (0 = System, 1 = User): 0  
  
ID      Type    AT    BT    CT    WT    TAT  
0       User    0     7     16    9     16  
1       System  1     5     6     0     5  
2       User    2     3     19    14    17  
3       System  3     4     10    3     7  
  
Average Waiting Time: 6.50  
Average Turnaround Time: 11.25  
  
Process returned 0 (0x0)  execution time : 57.110 s  
Press any key to continue.
```

Lab program 3:

Write a C program to simulate Real-Time CPU Scheduling algorithms: (Any one)

- a) Rate- Monotonic
- b) Earliest-deadline First
- c) Proportional scheduling

Code in C

```
#include <stdio.h>
#include <math.h>

void edf(int n, int
BT[], int DL[], int P[])
{
    int CT[n], TAT[n],
WT[n];
    int order[n];

    for (int i = 0; i < n;
i++)
        order[i] = i;

    for (int i = 0; i < n -
1; i++) {
        for (int j = i + 1; j
< n; j++) {
            if (DL[order[i]]
> DL[order[j]]) {
                int temp =
order[i];
                order[i] =
order[j];
                order[j] =
temp;
            }
        }
    }

    float U = 0;
    for (int i = 0; i < n;
i++)
        U += (float)BT[i]
/ P[i];

    printf("\nEarliest
Deadline First (EDF)
```

```

Scheduling\n");
    printf("CPU
Utilization: %.2f\n",
U);

    if (U <= 1)

printf("Schedulable
(Utilization <= 1)\n");
    else {
        printf("NOT
Schedulable
(Utilization > 1)\n");
        return;
    }

    int t = 0;

    for (int i = 0; i < n;
i++) {
        int idx = order[i];
        t += BT[idx];
        CT[idx] = t;
        TAT[idx] =
CT[idx];
        WT[idx] =
TAT[idx] - BT[idx];
    }

printf("ID\tBF\tDeadli
ne\tCT\tWT\tTAT\n");
    for (int i = 0; i < n;
i++) {

printf("%d\t%d\t%d\t\t
%d\t%d\t%d\n",
        i, BT[i], DL[i],
CT[i], WT[i], TAT[i]);
    }
}

void rms(int n, int
BT[], int DL[], int P[])
{
    int CT[n], TAT[n],
WT[n];

```

```

int order[n];

for (int i = 0; i < n;
i++)
    order[i] = i;

for (int i = 0; i < n -
1; i++) {
    for (int j = i + 1; j
< n; j++) {
        if (P[order[i]] >
P[order[j]]) {
            int temp =
order[i];
            order[i] =
order[j];
            order[j] =
temp;
        }
    }
}

float U = 0;
for (int i = 0; i < n;
i++)
    U += (float)BT[i]
/ P[i];

float bound = n *
(pow(2, 1.0/n) - 1);

printf("\n Rate
Monotonic Scheduling
(RMS) \n");
printf("CPU
Utilization: %.2f\n",
U);
printf("RM Bound:
%.4f\n", bound);

if (U <= bound)

printf("Schedulable
(Utilization <= RM
Bound)\n");
else
    printf("NOT

```

```

Schedulable
(Utilization > RM
Bound)\n");

    int t = 0;

    for (int i = 0; i < n;
i++) {
        int idx = order[i];
        t += BT[idx];
        CT[idx] = t;
        TAT[idx] =
CT[idx];
        WT[idx] =
TAT[idx] - BT[idx];
    }

    printf("ID\tBF\tPeriod\t
CT\tWT\tTAT\n");
    for (int i = 0; i < n;
i++) {

        printf("%d\t%d\t%d\t
%d\t%d\t%d\n",
            i, BT[i], P[i],
CT[i], WT[i], TAT[i]);
    }
}

void proportional(int
n, int BT[], int DL[],
int P[]) {
    int total = 0;

    for (int i = 0; i < n;
i++)
        total += P[i];

    printf("\n
Proportional
Scheduling \n");

    for (int i = 0; i < n;
i++) {
        float share =
(float)P[i] / total;

```



```

        printf("P%d ->
%.2f%%\n", i, share *
100);
    }
}

```

```

int main() {
    int n;
    printf("Enter
number of processes:
");
    scanf("%d", &n);

```

```

    int BT[n], DL[n],
P[n];

```

```

    printf("\nEnter
process details:\n");

```

```

    for (int i = 0; i < n;
i++) {
        printf("\nProcess
%d:\n", i);

```

```

        printf("Burst
Time: ");
        scanf("%d",
&BT[i]);

```

```

        printf("Deadline
(for EDF): ");
        scanf("%d",
&DL[i]);

```

```

        printf("Period
(for RMS): ");
        scanf("%d",
&P[i]);
    }

```

```

    edf(n, BT, DL, P);
    rms(n, BT, DL, P);
    proportional(n, BT,
DL, P);

```

```

    return 0;
}

```

OUTPUT

```
"C:\Users\Priyanka rajput\One  X + v
Enter number of processes: 3
Enter process details:
Process 0:
Burst Time: 3
Deadline (for EDF): 7
Period (for RMS): 20
Process 1:
Burst Time: 2
Deadline (for EDF): 4
Period (for RMS): 5
Process 2:
Burst Time: 2
Deadline (for EDF): 8
Period (for RMS): 10
Earliest Deadline First (EDF) Scheduling
CPU Utilization: 0.75
Schedulable (Utilization <= 1)
ID      BF      Deadline      CT      WT      TAT
0        3        7        5        2        5
1        2        4        2        0        2
2        2        8        7        5        7
Rate Monotonic Scheduling (RMS)
CPU Utilization: 0.75
RM Bound: 0.7798
Schedulable (Utilization <= RM Bound)
ID      BF      Period      CT      WT      TAT
0        3      20        7        4        7
1        2        5        2        0        2
2        2      10        4        2        4
Proportional Scheduling
P0 -> 57.14%
P1 -> 14.29%
P2 -> 28.57%
```

Lab program 4:

Write a C program to simulate: (Any one)

- a) Producer-Consumer problem using semaphores.
- b) Dining-Philosopher's problem

Code in C Producer Consumer

```
#include <stdio.h>

#include <stdlib.h>

#include <pthread.h>

#include <semaphore.h>

#include <unistd.h>


#define BUFFER_SIZE 5


int buffer[BUFFER_SIZE];

int in = 0, out = 0;


sem_t empty, full;

pthread_mutex_t mutex;


void* producer(void* arg) {

    int item;

    for(int i = 0; i < 7; i++) {

        item = i;

        sem_wait(&empty);

        pthread_mutex_lock(&mutex);
```

```

    buffer[in] = item;

    printf("Produced: %d at buffer[%d]\n", item, in);

    in = (in + 1) % BUFFER_SIZE;

    pthread_mutex_unlock(&mutex);

    sem_post(&full);

    sleep(1);
}
}

void* consumer(void* arg) {

    int item;

    for(int i = 0; i < 7; i++) {

        sem_wait(&full);

        pthread_mutex_lock(&mutex);

        item = buffer[out];

        printf("Consumed: %d from buffer[%d]\n", item, out);

        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&empty);

        sleep(2);

    }
}

```

```
}
```

```
int main() {
```

```
    pthread_t p, c;
```

```
    sem_init(&empty, 0, BUFFER_SIZE);
```

```
    sem_init(&full, 0, 0);
```

```
    pthread_mutex_init(&mutex, NULL);
```

```
    pthread_create(&p, NULL, producer, NULL);
```

```
    pthread_create(&c, NULL, consumer, NULL);
```

```
    pthread_join(p, NULL);
```

```
    pthread_join(c, NULL);
```

```
    sem_destroy(&empty);
```

```
    sem_destroy(&full);
```

```
    pthread_mutex_destroy(&mutex);
```

```
    return 0;
```

```
}
```

OUTPUT

Output

Clear

```
Produced: 0 at buffer[0]
Produced: 1 at buffer[1]
Produced: 2 at buffer[2]
Consumed: 0 from buffer[0]
Consumed: 1 from buffer[1]
Consumed: 2 from buffer[2]
Produced: 3 at buffer[3]
Produced: 4 at buffer[4]
Produced: 5 at buffer[0]
Consumed: 3 from buffer[3]
Consumed: 4 from buffer[4]
Consumed: 5 from buffer[0]
Produced: 6 at buffer[1]
Produced: 7 at buffer[2]
Produced: 8 at buffer[3]
Produced: 9 at buffer[4]
Consumed: 6 from buffer[1]
Consumed: 7 from buffer[2]
Consumed: 8 from buffer[3]
Consumed: 9 from buffer[4]
Produced: 10 at buffer[0]
Produced: 11 at buffer[1]
Consumed: 10 from buffer[0]
Consumed: 11 from buffer[1]
Produced: 12 at buffer[2]
Produced: 13 at buffer[3]
```

Code in C Dining Philosopher

```
#include <stdio.h>

#include <pthread.h>

#include <semaphore.h>

#include <unistd.h>


#define N 5


sem_t forks[N];


void* philosopher(void* num) {

    int id = *(int*)num;


    while (1) {

        printf("Philosopher %d is thinking.\n", id);

        sleep(1);


        if (id == N - 1) {

            // Prevent deadlock

            sem_wait(&forks[(id + 1) % N]);

            printf("Philosopher %d picked right fork %d.\n", id, (id + 1) % N);


            sem_wait(&forks[id]);

            printf("Philosopher %d picked left fork %d.\n", id, id);

        } else {

            sem_wait(&forks[id]);

            printf("Philosopher %d picked left fork %d.\n", id, id);

        }

    }

}
```

```

        sem_wait(&forks[(id + 1) % N]);

        printf("Philosopher %d picked right fork %d.\n", id, (id + 1) % N);
    }

    printf("Philosopher %d is eating.\n", id);

    sleep(1);

    sem_post(&forks[id]);

    sem_post(&forks[(id + 1) % N]);

    printf("Philosopher %d put down forks %d and %d.\n",
           id, id, (id + 1) % N);
}
}

int main() {
    pthread_t ph[N];
    int ids[N];

    for (int i = 0; i < N; i++)
        sem_init(&forks[i], 0, 1);

    for (int i = 0; i < N; i++) {
        ids[i] = i;

        pthread_create(&ph[i], NULL, philosopher, &ids[i]);
    }

    for (int i = 0; i < N; i++)
        pthread_join(ph[i], NULL);
}

```



```
    return 0;

}
```

OUTPUT

Output

```
Philosopher 0 is thinking.
Philosopher 1 is thinking.
Philosopher 4 is thinking.
Philosopher 2 is thinking.
Philosopher 3 is thinking.
Philosopher 0 picked left fork 0.
Philosopher 3 picked left fork 3.
Philosopher 3 picked right fork 4.
Philosopher 3 is eating.
Philosopher 1 picked left fork 1.
Philosopher 2 picked left fork 2.
Philosopher 3 put down forks 3 and 4.
Philosopher 3 is thinking.
Philosopher 2 picked right fork 3.
Philosopher 2 is eating.
Philosopher 2 put down forks 2 and 3.
Philosopher 2 is thinking.
Philosopher 3 picked left fork 3.
Philosopher 1 picked right fork 2.
Philosopher 1 is eating.
Philosopher 3 picked right fork 4.
Philosopher 3 is eating.
Philosopher 1 put down forks 1 and 2.
Philosopher 1 is thinking.
Philosopher 2 picked left fork 2.
Philosopher 0 picked right fork 1.
```

Lab program 5:

Write a C program to simulate: (Any one)

- a) Bankers' algorithm for the purpose of deadlock avoidance.
- b) Deadlock Detection

Code in C Banker's algorithm

```
#include <stdio.h>
```

```
int main() {
    int n, m, i, j, k;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resource types: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m];
    int avail[m];

    // Allocation Matrix
    printf("\nEnter Allocation Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }

    // Maximum Matrix
    printf("\nEnter Maximum Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            scanf("%d", &max[i][j]);
        }
    }

    // Available Resources
    printf("\nEnter Available Resources:\n");
    for(i = 0; i < m; i++) {
        scanf("%d", &avail[i]);
    }

    // Calculate Need Matrix
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }

    // Display Need Matrix
    printf("\nNeed Matrix:\n");
    for(i = 0; i < n; i++) {
```

```

    for(j = 0; j < m; j++) {
        printf("%d ", need[i][j]);
    }
    printf("\n");
}

// ----- SAFETY ALGORITHM -----

int finish[n], safeSeq[n];
int work[m];

for(i = 0; i < m; i++)
    work[i] = avail[i];

for(i = 0; i < n; i++)
    finish[i] = 0;

int count = 0;

while(count < n) {
    int found = 0;

    for(i = 0; i < n; i++) {

        if(finish[i] == 0) {

            for(j = 0; j < m; j++) {
                if(need[i][j] > work[j])
                    break;
            }

            if(j == m) {

                for(k = 0; k < m; k++)
                    work[k] += alloc[i][k];

                safeSeq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }

    if(found == 0)
        break;
}

if(count == n) {
    printf("\nSystem is in SAFE state.\n");
    printf("Safe Sequence: ");

    for(i = 0; i < n; i++) {
        printf("P%d", safeSeq[i]);
    }
}

```

```

        if(i != n - 1)
            printf(" -> ");
    }

    printf("\n");
}
else {
    printf("\nSystem is NOT in safe state.\n");
    return 0;
}

// ----- RESOURCE REQUEST ALGORITHM -----

int p;
int request[m];

printf("\nEnter process number making request: ");
scanf("%d", &p);

printf("Enter request for resources:\n");
for(i = 0; i < m; i++) {
    scanf("%d", &request[i]);
}

// Check Request <= Need
for(i = 0; i < m; i++) {
    if(request[i] > need[p][i]) {
        printf("\nError: Process has exceeded maximum claim.\n");
        return 0;
    }
}

// Check Request <= Available
for(i = 0; i < m; i++) {
    if(request[i] > avail[i]) {
        printf("\nResources not available. Process must wait.\n");
        return 0;
    }
}

// Pretend allocation
for(i = 0; i < m; i++) {
    avail[i] -= request[i];
    alloc[p][i] += request[i];
    need[p][i] -= request[i];
}

// Run Safety Algorithm Again
for(i = 0; i < m; i++)
    work[i] = avail[i];

for(i = 0; i < n; i++)

```

```

    finish[i] = 0;

count = 0;

while(count < n) {
    int found = 0;

    for(i = 0; i < n; i++) {

        if(finish[i] == 0) {

            for(j = 0; j < m; j++) {
                if(need[i][j] > work[j])
                    break;
            }

            if(j == m) {

                for(k = 0; k < m; k++)
                    work[k] += alloc[i][k];

                safeSeq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }

    if(found == 0)
        break;
}

if(count == n) {

    printf("\nRequest can be granted.\n");
    printf("New Safe Sequence: ");

    for(i = 0; i < n; i++) {
        printf("P%d", safeSeq[i]);

        if(i != n - 1)
            printf(" -> ");
    }

    printf("\n");
}
else {

    printf("\nRequest cannot be granted. System will enter unsafe state.\n");
}

return 0;
}

```

OUTPUT

```
"C:\Users\Priyanka rajput\One... X + v
Enter number of processes: 5
Enter number of resource types: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Maximum Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resources:
3 3 2

Need Matrix:
7 4 3
1 2 2
6 0 0
0 1 1
4 3 1

System is in SAFE state.
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2

Enter process number making request: 1
Enter request for resources:
1 0 2

Request can be granted.
New Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2

Process returned 0 (0x0)    execution time : 61.781 s
Press any key to continue.
```

Code in C Deadlock

```
#include <stdio.h>

int main() {
    int n, m, i, j, k;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resource types: ");
    scanf("%d", &m);

    int alloc[n][m], request[n][m];
    int avail[m];

    // Allocation Matrix
    printf("\nEnter Allocation Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }

    // Request Matrix
    printf("\nEnter Request Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            scanf("%d", &request[i][j]);
        }
    }

    // Available Resources
    printf("\nEnter Available Resources:\n");
    for(i = 0; i < m; i++) {
        scanf("%d", &avail[i]);
    }

    int finish[n];

    // Initialize finish array
    for(i = 0; i < n; i++) {

        int sum = 0;

        for(j = 0; j < m; j++)
            sum += alloc[i][j];

        if(sum == 0)
            finish[i] = 1;
        else
            finish[i] = 0;
    }
}
```

```

int work[m];

for(i = 0; i < m; i++)
    work[i] = avail[i];

int found;

do {
    found = 0;

    for(i = 0; i < n; i++) {

        if(finish[i] == 0) {

            for(j = 0; j < m; j++) {

                if(request[i][j] > work[j])
                    break;
            }

            // Process can finish
            if(j == m) {

                for(k = 0; k < m; k++)
                    work[k] += alloc[i][k];

                finish[i] = 1;
                found = 1;

                printf("\nP%d can finish.", i);
            }
        }
    }
} while(found);

// Check deadlock
int deadlock = 0;

printf("\n\nProcesses in Deadlock:\n");

for(i = 0; i < n; i++) {

    if(finish[i] == 0) {
        printf("P%d ", i);
        deadlock = 1;
    }
}

if(deadlock == 0)
    printf("No deadlock detected. All processes can finish.");

return 0;

```


}

OUTPUT

```
"C:\Users\Priyanka rajput\One" X + v
Enter number of resource types: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2

Enter Request Matrix:
0 0 0
2 0 2
0 0 0
1 0 1
0 0 2

Enter Available Resources:
0 0 0

P0 can finish.
P2 can finish.
P3 can finish.
P4 can finish.
P1 can finish.

Processes in Deadlock:
No deadlock detected. All processes can finish.
Process returned 0 (0x0)    execution time : 67.833 s
Press any key to continue.
```

Lab program 6:

Write a C program to simulate the following contiguous memory allocation techniques. (Any one)

- a) Worst-fit
- b) Best-fit
- c) First-fit.

Code in C

```
#include <stdio.h>

void firstFit(int blocks[], int m, int process[], int n) {
    int allocation[n];
    int used[m];

    for(int i = 0; i < n; i++)
        allocation[i] = -1;

    for(int i = 0; i < m; i++)
        used[i] = 0;

    for(int i = 0; i < n; i++) {
        for(int j = 0; j < m; j++) {
            if(blocks[j] >= process[i] && used[j] == 0) {
                allocation[i] = j;
                used[j] = 1;
                break;
            }
        }
    }

    printf("\n--- First Fit ---\n");
}
```

```

printf("Process No.\tProcess Size\tBlock No.\n");

for(int i = 0; i < n; i++) {
    printf("%d\t%d\t", i + 1, process[i]);

    if(allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}
}

void bestFit(int blocks[], int m, int process[], int n) {
    int allocation[n];
    int used[m];

    for(int i = 0; i < n; i++)
        allocation[i] = -1;

    for(int i = 0; i < m; i++)
        used[i] = 0;

    for(int i = 0; i < n; i++) {
        int best = -1;

        for(int j = 0; j < m; j++) {
            if(blocks[j] >= process[i] && used[j] == 0) {
                if(best == -1 || blocks[j] < blocks[best])

```

```

        best = j;
    }
}

if(best != -1) {
    allocation[i] = best;
    used[best] = 1;
}
}

printf("\n--- Best Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");

for(int i = 0; i < n; i++) {
    printf("%d\t%d\t", i + 1, process[i]);

    if(allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}
}

void worstFit(int blocks[], int m, int process[], int n) {
    int allocation[n];
    int used[m];

    for(int i = 0; i < n; i++)

```

```

    allocation[i] = -1;

for(int i = 0; i < m; i++)
    used[i] = 0;

for(int i = 0; i < n; i++) {
    int worst = -1;

    for(int j = 0; j < m; j++) {
        if(blocks[j] >= process[i] && used[j] == 0) {
            if(worst == -1 || blocks[j] > blocks[worst])
                worst = j;
        }
    }

    if(worst != -1) {
        allocation[i] = worst;
        used[worst] = 1;
    }
}

printf("\n--- Worst Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");

for(int i = 0; i < n; i++) {
    printf("%d\t%d\t", i + 1, process[i]);

    if(allocation[i] != -1)

```

```

        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
    }
}

int main() {
    int m, n;

    printf("Enter number of memory blocks: ");
    scanf("%d", &m);

    int blocks[m];

    printf("Enter sizes of %d memory blocks:\n", m);

    for(int i = 0; i < m; i++)
        scanf("%d", &blocks[i]);

    printf("\nEnter number of processes: ");
    scanf("%d", &n);

    int process[n];

    printf("Enter sizes of %d processes:\n", n);

    for(int i = 0; i < n; i++)
        scanf("%d", &process[i]);

```

```

firstFit(blocks, m, process, n);

bestFit(blocks, m, process, n);

worstFit(blocks, m, process, n);

return 0;
}

```

OUTPUT

```

"C:\Users\Priyanka rajput\One... x + v
Enter number of memory blocks: 6
Enter sizes of 6 memory blocks:
300 600 350 200 750 125

Enter number of processes: 5
Enter sizes of 5 processes:
115 500 358
200 375

--- First Fit ---
Process No.      Process Size      Block No.
1                115              1
2                500              2
3                358              5
4                200              3
5                375              Not Allocated

--- Best Fit ---
Process No.      Process Size      Block No.
1                115              6
2                500              2
3                358              5
4                200              4
5                375              Not Allocated

--- Worst Fit ---
Process No.      Process Size      Block No.
1                115              5
2                500              2
3                358              Not Allocated
4                200              3
5                375              Not Allocated

Process returned 0 (0x0)   execution time : 31.083 s
Press any key to continue.
|

```

Lab program 7:

Write a C program to simulate page replacement algorithms. (Any one)

- a) FIFO
- b) LRU
- c) Optimal

Code in C

```
#include <stdio.h>

int search(int frames[], int f, int key)
{
    for(int i = 0; i < f; i++)
    {
        if(frames[i] == key)
            return 1;
    }
    return 0;
}

/* ----- FIFO ----- */

void fifo(int pages[], int n, int f)
{
    int frames[10], index = 0, faults = 0;

    for(int i = 0; i < f; i++)
        frames[i] = -1;

    printf("\n\n--- FIFO Page Replacement ---\n");

    for(int i = 0; i < n; i++)
    {
        if(!search(frames, f, pages[i]))
        {
            frames[index] = pages[i];
            index = (index + 1) % f;
            faults++;
        }

        printf("\nPage %d --> ", pages[i]);

        for(int j = 0; j < f; j++)
        {
            if(frames[j] != -1)
                printf("%d ", frames[j]);
            else
                printf("- ");
        }
    }
}
```



```

    printf("\n\nTotal Page Faults = %d\n", faults);
}

/* ----- LRU ----- */

void lru(int pages[], int n, int f)
{
    int frames[10], time[10];
    int faults = 0, count = 0;

    for(int i = 0; i < f; i++)
    {
        frames[i] = -1;
        time[i] = 0;
    }

    printf("\n\n--- LRU Page Replacement ---\n");

    for(int i = 0; i < n; i++)
    {
        int flag = 0;

        for(int j = 0; j < f; j++)
        {
            if(frames[j] == pages[i])
            {
                count++;
                time[j] = count;
                flag = 1;
                break;
            }
        }

        if(flag == 0)
        {
            int pos = 0;

            for(int j = 1; j < f; j++)
            {
                if(time[j] < time[pos])
                    pos = j;
            }

            count++;
            frames[pos] = pages[i];
            time[pos] = count;
            faults++;
        }

        printf("\nPage %d --> ", pages[i]);

        for(int j = 0; j < f; j++)

```

```

        {
            if(frames[j] != -1)
                printf("%d ", frames[j]);
            else
                printf("- ");
        }
    }

    printf("\n\nTotal Page Faults = %d\n", faults);
}

/* ----- Optimal ----- */

void optimal(int pages[], int n, int f)
{
    int frames[10];
    int faults = 0;

    for(int i = 0; i < f; i++)
        frames[i] = -1;

    printf("\n\n--- Optimal Page Replacement ---\n");

    for(int i = 0; i < n; i++)
    {
        if(search(frames, f, pages[i]))
        {
            printf("\nPage %d --> ", pages[i]);

            for(int j = 0; j < f; j++)
            {
                if(frames[j] != -1)
                    printf("%d ", frames[j]);
                else
                    printf("- ");
            }

            continue;
        }

        int pos = -1, farthest = i + 1;

        for(int j = 0; j < f; j++)
        {
            int k;

            for(k = i + 1; k < n; k++)
            {
                if(frames[j] == pages[k])
                {
                    if(k > farthest)
                    {
                        farthest = k;
                    }
                }
            }
        }
    }
}

```

```

        pos = j;
    }
    break;
}
}

if(k == n)
{
    pos = j;
    break;
}
}

if(pos == -1)
    pos = 0;

frames[pos] = pages[i];
faults++;

printf("\nPage %d --> ", pages[i]);

for(int j = 0; j < f; j++)
{
    if(frames[j] != -1)
        printf("%d ", frames[j]);
    else
        printf("- ");
}
}

printf("\n\nTotal Page Faults = %d\n", faults);
}

/* ----- Main Function ----- */

int main()
{
    int pages[50], n, f, choice;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter page reference string:\n");

    for(int i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    printf("Enter number of frames: ");
    scanf("%d", &f);

    printf("\nChoose Page Replacement Technique");
    printf("\n1. FIFO");
    printf("\n2. LRU");

```

```
printf("\n3. Optimal");
printf("\n4. All");

printf("\n\nEnter your choice: ");
scanf("%d", &choice);

switch(choice)
{
    case 1:
        fifo(pages, n, f);
        break;

    case 2:
        lru(pages, n, f);
        break;

    case 3:
        optimal(pages, n, f);
        break;

    case 4:
        fifo(pages, n, f);
        lru(pages, n, f);
        optimal(pages, n, f);
        break;

    default:
        printf("Invalid Choice");
}

return 0;
}
```

OUTPUT

```
"C:\Users\Priyanka rajput\One" X + v
Enter number of pages: 12
Enter page reference string:
1 2 3 4 1 2 5 1 2 3 4 5
Enter number of frames: 3

Choose Page Replacement Technique
1. FIFO
2. LRU
3. Optimal
4. All

Enter your choice: 4

--- FIFO Page Replacement ---

Page 1 --> 1 - -
Page 2 --> 1 2 -
Page 3 --> 1 2 3
Page 4 --> 4 2 3
Page 1 --> 4 1 3
Page 2 --> 4 1 2
Page 5 --> 5 1 2
Page 1 --> 5 1 2
Page 2 --> 5 1 2
Page 3 --> 5 3 2
Page 4 --> 5 3 4
Page 5 --> 5 3 4

Total Page Faults = 9
```

```
"C:\Users\Priyanka rajput\One" X + v

--- LRU Page Replacement ---

Page 1 --> 1 - -
Page 2 --> 1 2 -
Page 3 --> 1 2 3
Page 4 --> 4 2 3
Page 1 --> 4 1 3
Page 2 --> 4 1 2
Page 5 --> 5 1 2
Page 1 --> 5 1 2
Page 2 --> 5 1 2
Page 3 --> 3 1 2
Page 4 --> 3 4 2
Page 5 --> 3 4 5

Total Page Faults = 10

--- Optimal Page Replacement ---

Page 1 --> 1 - -
Page 2 --> 1 2 -
Page 3 --> 1 2 3
Page 4 --> 1 2 4
Page 1 --> 1 2 4
Page 2 --> 1 2 4
Page 5 --> 1 2 5
Page 1 --> 1 2 5
Page 2 --> 1 2 5
Page 3 --> 3 2 5
Page 4 --> 4 2 5
Page 5 --> 4 2 5

Total Page Faults = 7

Process returned 0 (0x0) execution time : 25.282 s
Press any key to continue.
```